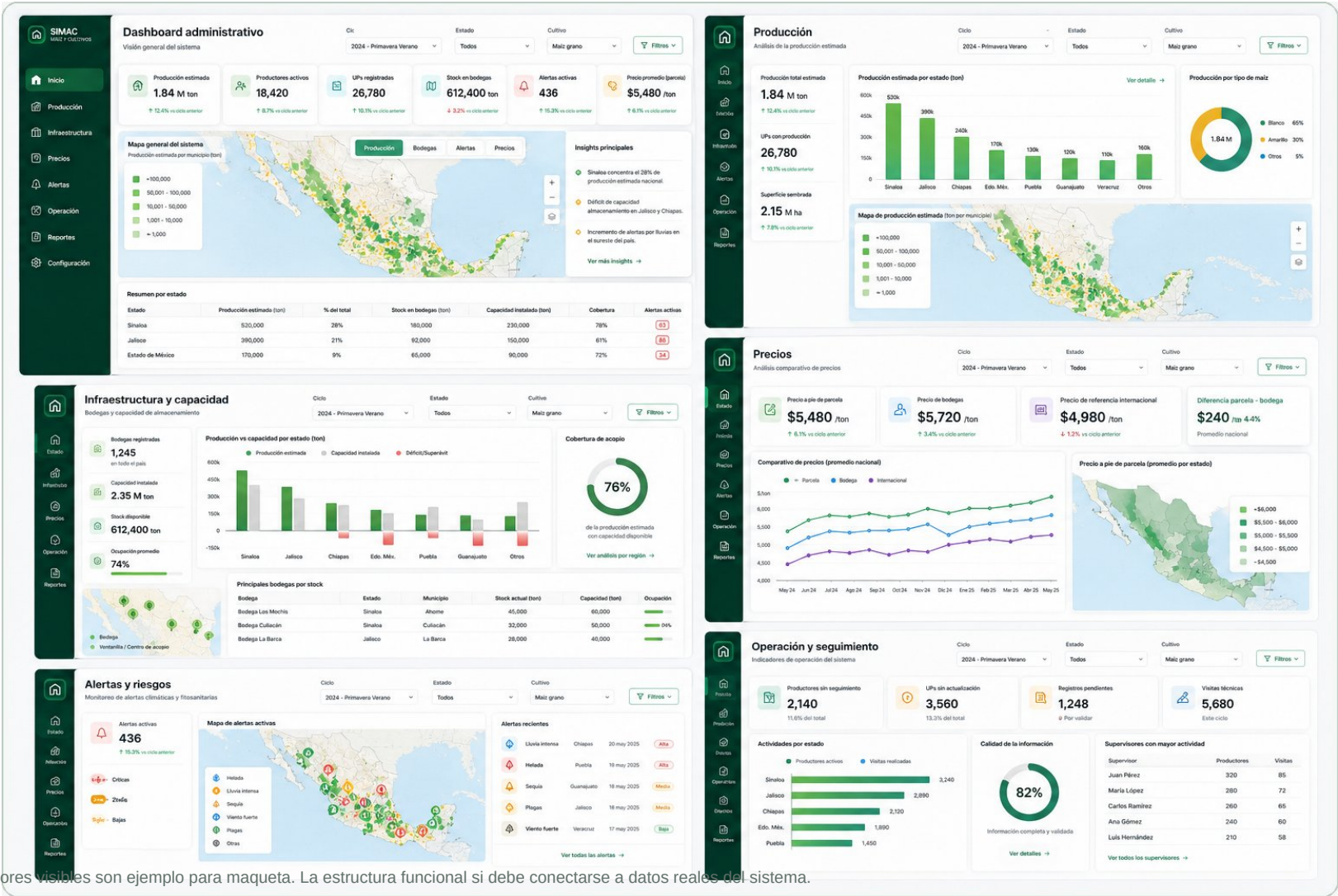


Dashboard Administrador - Sistema Maiz

Manual visual y funcional para desarrollo

Este documento explica, pantalla por pantalla, que debe construir el programador, de donde sale cada dato del desarrollo y como se integran productores, bodegas, precios, alertas y seguimiento en un tablero para toma de decisiones.



Importante: los valores visibles son ejemplo para maqueta. La estructura funcional si debe conectarse a datos reales del sistema.

1. Logica general del dashboard

El administrador no captura informacion diaria. El administrador visualiza, compara, filtra y detecta prioridades. El dashboard transforma registros de productores, bodegueros, supervisores y sistema automatico en informacion ejecutiva.

Origen	Que registra	Uso en dashboard
Productor / sembrador	UPS, poligonos, ciclos, cultivos, seguimiento, incidencias y precio a pie de parcela	Produccion, ubicacion, avance, alertas y precio de campo
Bodeguero	Bodegas, capacidad, inventarios y precio de bodega	Stock, capacidad, ocupacion, cobertura de acopio y precio bodega
Admin	Precio de referencia internacional, usuarios y permisos	Comparativo de mercado y control global
Sistema automatico	Alertas climaticas por Open-Meteo y reglas de riesgo	Alertas territoriales, severidad y notificaciones
Supervisor	Revision, validacion y solicitudes de correccion	Calidad de informacion, pendientes y seguimiento operativo

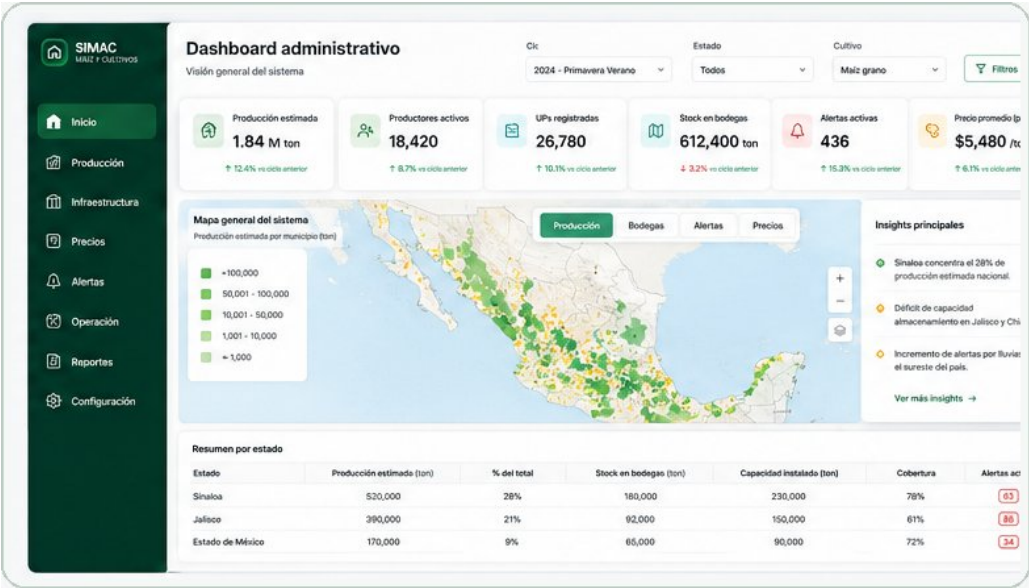
Reglas globales que deben respetarse

- Todos los filtros globales deben afectar KPIs, mapas, graficas y tablas: estado, municipio, ciclo, cultivo y estatus.
- El mapa es el elemento principal. Cada pantalla debe tener una lectura territorial, no solo tablas.
- El backend debe entregar datos agregados para dashboard. El frontend no debe calcular todo desde registros crudos masivos.
- Los datos sin captura real se muestran como N/D, pendiente o 0, pero no se inventan en produccion.
- Solo existen tres precios: precio a pie de parcela, precio de bodega y precio de referencia internacional.

Tipo de precio	Quien lo genera	Uso
Precio a pie de parcela	Productor / captura de campo	Mide cuanto recibe el productor en territorio
Precio de bodega	Bodeguero	Mide condiciones de acopio y compra
Precio internacional	Admin	Referencia global para comparativo

2. Pantalla: Vision general del sistema

Pantalla de entrada. Debe dar una lectura rapida: cuanto se produce, donde esta la produccion, cuanto stock hay, que alertas existen y como esta el precio a pie de parcela.



Que debe verse en pantalla

Elemento	Funcion
KPIs superiores	Produccion estimada, productores, UPs, stock, alertas y precio parcela.
Mapa general	Capas: produccion, bodegas, alertas y precios.
Insights	Prioridades: deficit, alertas, territorios criticos.
Tabla resumen	Estado, produccion, stock, cobertura y alertas.

De donde sale la informacion del desarrollo

Dato	Origen	Regla
Productores	usuarios/productores	Contar productores con UP o ciclo activo.
UPs	unidades_produccion	Contar UPs filtradas por territorio/ciclo.
Produccion	estimaciones/cultivos	Sumar estimaciones reales registradas.
Stock	inventarios_bodega	Sumar inventario actual de bodegas.
Alertas	alertas	Contar activas o pendientes.
Precio parcela	precios parcela	Promedio por territorio, cultivo y fecha.

Implementacion - Vision general del sistema

Esta pagina traduce la pantalla anterior a tareas concretas de frontend, backend y calculos. El programador debe construir primero con datos mock y despues conectar la API real.

Pasos concretos para frontend

- Crear DashboardAdmin.vue con sidebar, filtros, KPIGrid, DecisionMap, InsightsPanel y tabla.
- Maquetar primero con datos mock.
- Conectar filtros globales a toda la pantalla.
- El mapa debe recibir GeoJSON de UPS, bodegas y alertas.

KPIs o calculos clave

- Produccion estimada = $\text{SUM}(\text{estimacion_toneladas})$.
- Stock = $\text{SUM}(\text{inventario_actual_ton})$.
- Cobertura = $\text{capacidad_disponible} / \text{produccion_estimada}$.
- Alertas = $\text{COUNT}(\text{alertas activas o pendientes})$.

API / backend esperado

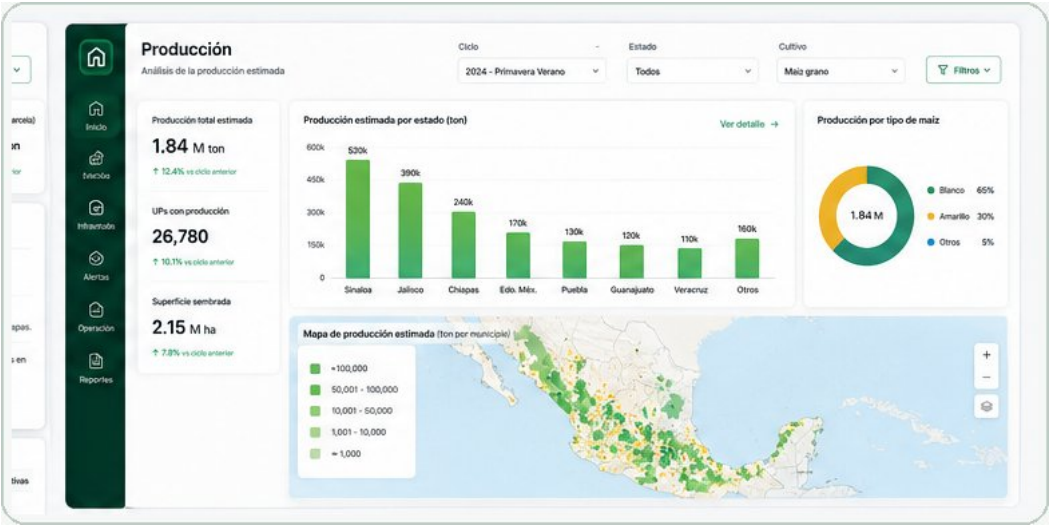
- GET /dashboard/admin/resumen
- GET /dashboard/admin/mapa
- GET /dashboard/admin/tabla-estados
- La API debe devolver totales agregados.

3. Pantalla: Produccion

Responde donde se esta produciendo maiz, cuanto se estima cosechar y que tipo de maiz concentra el volumen. Sirve para priorizar territorios productivos.

Que debe verse en pantalla

Elemento	Funcion
KPIs	Produccion total, UPs con produccion, superficie sembrada.
Barras	Produccion estimada por estado/municipio.
Dona	Participacion por tipo de maiz.
Mapa calor	Intensidad territorial de produccion.



De donde sale la informacion del desarrollo

Dato	Origen	Regla
Superficie	cycle_crops	SUM(superficie_sembrada).
Produccion	estimaciones	SUM(estimacion_toneladas).
Tipo maiz	catalogo cultivo	Agrupar por tipo definido.
Mapa	UPs + poligonos	Pintar por toneladas estimadas.
Avance	seguimientos	Usar ultimo seguimiento por cultivo.

Implementacion - Produccion

Esta pagina traduce la pantalla anterior a tareas concretas de frontend, backend y calculos. El programador debe construir primero con datos mock y despues conectar la API real.

Pasos concretos para frontend

- Crear pantalla o tab Produccion.
- La barra debe venir agregada por backend.
- El mapa debe permitir zoom y click a detalle.
- No mezclar ciclos: todo depende del filtro ciclo.

KPIs o calculos clave

- UPs con produccion = COUNT(DISTINCT up_id con cultivo en ciclo).
- Superficie sembrada = SUM(superficie_sembrada).
- Produccion estimada = SUM(estimacion_toneladas).

API / backend esperado

- GET /dashboard/admin/produccion/resumen
- GET /dashboard/admin/produccion/por-territorio
- GET /dashboard/admin/produccion/por-tipo-maiz
- GET /dashboard/admin/produccion/mapa.geojson

4. Pantalla: Infraestructura y capacidad

Cruza produccion esperada contra bodegas, capacidad e inventario. Sirve para saber si el territorio puede acopiar lo que se va a producir.

Que debe verse en pantalla



Elemento	Funcion
KPIs	Bodegas, capacidad, stock y ocupacion.
Comparativo	Produccion estimada vs capacidad instalada.
Cobertura	Porcentaje de produccion con capacidad disponible.
Mapa	Bodegas, ventanillas y centros de acopio.
Tabla	Bodegas por stock, capacidad y ocupacion.

De donde sale la informacion del desarrollo

Dato	Origen	Regla
Bodegas	infraestructura	Contar activos con funcion bodega.
Capacidad	bodegas	SUM(capacidad_total_ton).
Stock	inventarios	SUM(inventario_actual_ton).
Ocupacion	inventarios/capacidad	stock / capacidad.
Deficit	produccion + bodegas	capacidad disponible - produccion estimada.
Cobertura	PostGIS/municipio	MVP por municipio; futuro por distancia.

Implementacion - Infraestructura y capacidad

Esta pagina traduce la pantalla anterior a tareas concretas de frontend, backend y calculos. El programador debe construir primero con datos mock y despues conectar la API real.

Pasos concretos para frontend

- Crear InfrastructurePanel.vue.
- En MVP cruzar por estado/municipio.
- Fase posterior: cruce espacial por radio con PostGIS.
- Marcar deficit en rojo.
- Admin solo visualiza, no captura bodegas.

KPIs o calculos clave

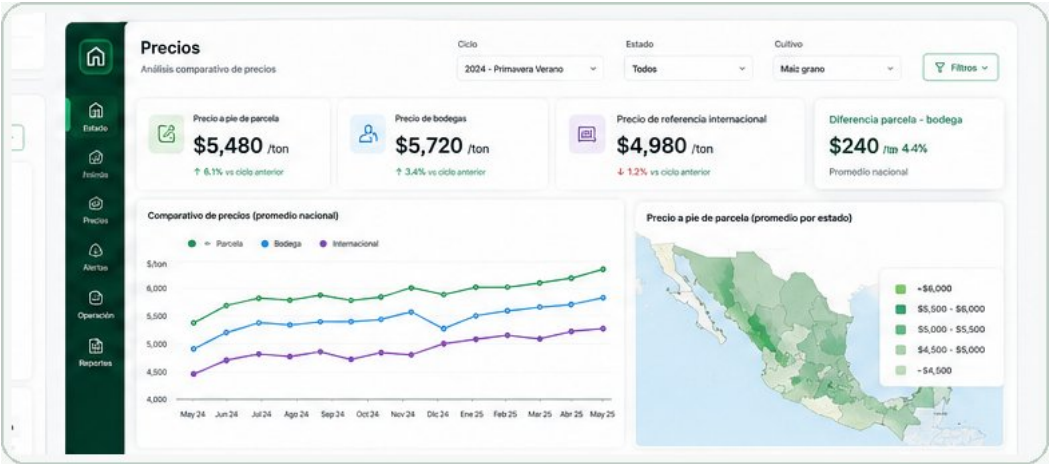
- Deficit = capacidad_disponible - produccion_estimada.
- Ocupacion = $\text{SUM}(\text{stock_actual}) / \text{SUM}(\text{capacidad_total})$.
- Cobertura = capacidad_disponible / produccion_estimada.

API / backend esperado

- GET /dashboard/admin/infraestructura/resumen
- GET /dashboard/admin/infraestructura/produccion-vs-capacidad
- GET /dashboard/admin/infraestructura/bodegas.geojson
- GET /dashboard/admin/infraestructura/top-bodegas

5. Pantalla: Precios

Compara los tres precios oficiales del sistema: precio a pie de parcela, precio de bodega y precio de referencia internacional. Muestra brecha territorial y tendencia historica.



Que debe verse en pantalla

Elemento	Funcion
Cards	Precio actual/promedio de parcela, bodega e internacional.
Brecha	Diferencia entre parcela y bodega.
Linea historica	Tendencia de los tres precios.
Mapa	Precio a pie de parcela o brecha por territorio.

De donde sale la informacion del desarrollo

Dato	Origen	Regla
Parcela	precios tipo parcela	Lo registra productor/campo.
Bodega	precios tipo bodega	Lo registra bodeguero.
Internacional	precios tipo internacional	Lo registra admin.
Brecha	precios	precio_bodega - precio_parcela.
Historico	precios.fecha	Agrupar por semana o mes.

Implementacion - Precios

Esta pagina traduce la pantalla anterior a tareas concretas de frontend, backend y calculos. El programador debe construir primero con datos mock y despues conectar la API real.

Pasos concretos para frontend

- Usar tabla unica precios con tipo_precio.
- No crear precio de gobierno ni otros tipos.
- El mapa alterna parcela, bodega o brecha.
- Validar permisos por rol en frontend y backend.

KPIs o calculos clave

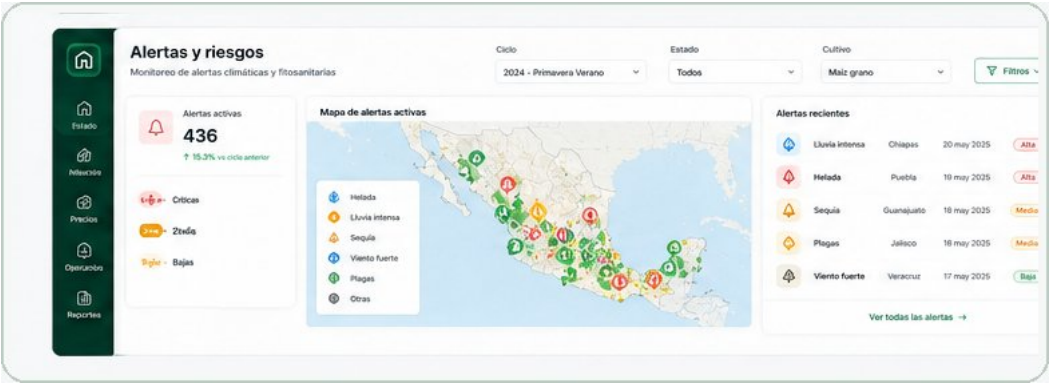
- Precio parcela = $AVG(valor \text{ WHERE } tipo_precio=parcela)$.
- Precio bodega = $AVG(valor \text{ WHERE } tipo_precio=bodega)$.
- Precio internacional = ultimo valor o promedio del periodo.
- Brecha = $AVG(bodega) - AVG(parcela)$.

API / backend esperado

- GET /dashboard/admin/precios/resumen
- GET /dashboard/admin/precios/historico
- GET /dashboard/admin/precios/mapa
- POST /precios solo para roles permitidos

6. Pantalla: Alertas y riesgos

Muestra riesgos climaticos y productivos para actuar antes de perder produccion. Debe separar alertas automaticas y manuales, y priorizar por severidad.



Que debe verse en pantalla

Elemento	Funcion
KPIs	Total activas, criticas, medias y bajas.
Mapa	Iconos por ubicacion y color por severidad.
Leyenda	Helada, lluvia intensa, viento, plagas, otras.
Lista	Alertas recientes priorizadas.

De donde sale la informacion del desarrollo

Dato	Origen	Regla
Automaticas	Open-Meteo/alertas	Reglas cada 12 o 24 horas.
Manuales	reportes productor	Selector UP, ciclo y cultivo; no IDs.
Severidad	alertas	Critica, media o baja.
Ubicacion	UP/centroid	Mostrar punto o territorio.
Estatus	alertas	Activa, pendiente, validada, resuelta.

Implementacion - Alertas y riesgos

Esta pagina traduce la pantalla anterior a tareas concretas de frontend, backend y calculos. El programador debe construir primero con datos mock y despues conectar la API real.

Pasos concretos para frontend

- Crear AlertsPanel.vue.
- Iconos por tipo y color por severidad.
- Evitar duplicados por UP, tipo y periodo.
- Filtros por tipo, severidad, estado y fecha.
- Click abre detalle de UP/cultivo/ciclo.

KPIs o calculos clave

- Alertas activas = COUNT(estatus activa/pendiente).
- Criticas = COUNT(severidad critica).
- UPs afectadas = COUNT(DISTINCT up_id).

API / backend esperado

- GET /dashboard/admin/alertas/resumen
- GET /dashboard/admin/alertas/mapa.geojson
- GET /dashboard/admin/alertas/lista
- GET /alertas/:id

7. Pantalla: Operacion y seguimiento

Mide si la informacion del sistema es confiable y actualizada. No solo mide maiz: mide calidad de captura, revision y seguimiento.

Que debe verse en pantalla



Elemento	Funcion
KPIs	Sin seguimiento, sin actualizacion, pendientes y visitas.
Barras	Actividad por estado.
Calidad	Porcentaje de datos completos y validados.
Ranking	Supervisores con mayor actividad.

De donde sale la informacion del desarrollo

Dato	Origen	Regla
Sin seguimiento	productores/seguimientos	UP activa sin seguimiento reciente.
Sin actualizacion	UPs/seguimientos	Ultimo seguimiento fuera de umbral.
Pendientes	validaciones	Registros por revisar o corregir.
Calidad	campos obligatorios	Registros completos / totales.
Supervisor	supervisor_productor	Productores vinculados y revisiones.

Implementacion - Operacion y seguimiento

Esta pagina traduce la pantalla anterior a tareas concretas de frontend, backend y calculos. El programador debe construir primero con datos mock y despues conectar la API real.

Pasos concretos para frontend

- Crear OperationsPanel.vue.
- Definir umbral configurable, por ejemplo 30 dias.
- Mostrar pendientes por estado.
- Separar de dashboard productor: esto es agregado.
- Debe detectar donde falta captura o validacion.

KPIs o calculos clave

- Calidad = $\text{registros_completos} / \text{registros_totales}$.
- Sin seguimiento = COUNT(UPs sin seguimiento dentro del umbral).
- Pendientes = COUNT(validaciones pendientes).

API / backend esperado

- GET /dashboard/admin/operacion/resumen
- GET /dashboard/admin/operacion/actividad-por-estado
- GET /dashboard/admin/operacion/calidad
- GET /dashboard/admin/operacion/supervisores



8. Orden exacto de implementacion

No intentar conectar todo al mismo tiempo. Primero UI con datos de prueba, despues endpoints agregados y al final datos reales. Asi se valida con la Subsecretaria sin esperar toda la integracion.

Fase	Que debe hacer	Resultado
1. UI mock	Construir sidebar, filtros, cards, mapas simulados, graficas y tablas con datos de prueba.	La pantalla se ve como producto real.
2. Endpoints	Crear /dashboard/admin/* con resúmenes agregados.	El frontend consume datos limpios.
3. Mapas	Conectar GeoJSON de UPs, bodegas y alertas.	Mapa filtrable y territorial.
4. Precios	Implementar tabla unica con tipo_precio.	Comparativo correcto de tres precios.
5. Validacion	Comparar totales dashboard vs base real.	Indicadores confiables.

Reglas de trabajo para el agente de programacion

- Primero debe replicar la maqueta visual con datos mock.
- Despues debe crear endpoints agregados para cada pantalla.
- Finalmente debe conectar esos endpoints a los componentes reales.
- Cada pantalla debe poder operar con filtros globales.
- Cada indicador debe tener una formula o regla de origen clara.



9. Componentes y errores a evitar

Componentes sugeridos en frontend

Componente	Responsabilidad
AdminDashboardLayout.vue	Sidebar, header y contenedor de pantallas.
GlobalFilters.vue	Estado, municipio, ciclo, cultivo, estatus y fecha.
KPIGrid.vue	Tarjetas superiores de indicadores.
DecisionMap.vue	Mapa con capas de produccion, bodegas, alertas y precios.
ProductionPanel.vue	Graficas y mapa de produccion.
InfrastructurePanel.vue	Produccion vs capacidad, bodegas y stock.
PricesPanel.vue	Tres precios, brecha e historico.
AlertsPanel.vue	Mapa y lista de alertas.
OperationsPanel.vue	Calidad de informacion y seguimiento.

Errores que debe evitar el programador

- No convertir el dashboard admin en pantalla de captura. Es consulta y decision.
- No crear mas tipos de precio. Solo parcela, bodega e internacional.
- No pintar mapas sin coordenadas o GeoJSON.
- No dejar filtros que solo afecten una parte. Deben afectar todo.
- No inventar totales cuando no existan registros. Mostrar N/D, pendiente o 0 segun corresponda.
- No calcular todo desde el frontend con registros crudos. El backend debe mandar resúmenes agregados.